

Classical Solution for Real-World Supply Chain Route Optimization

A Step-by-Step Guide Using Python and Google OR-Tools

October 18, 2025

1. Project Overview

The objective of this project is to develop and compare classical and quantum solutions for a real-world supply chain route optimization problem. This document details the first major phase: the creation of a robust and realistic **classical benchmark**.

We will define a complex routing problem, ground it with real-world data and constraints, and solve it using Google's industry-standard OR-Tools library. This solution will serve as the definitive baseline against which a future quantum algorithm will be evaluated.

2. Problem Definition: The CVRPTW

The problem we are solving is a **Capacitated Vehicle Routing Problem with Time Windows (CVRPTW)**. It is a well-known optimization problem that closely mirrors real-world logistics challenges.

Entities

- **A single depot:** The central warehouse where all vehicles start and end their routes.
- **N customers:** A set of locations to be served, each with a specific demand for goods.
- **K vehicles:** A fleet of delivery vehicles, each with a known maximum cargo capacity.

Constraints & Objectives

1. **Objective: Minimize Total Distance:** The primary goal is to find the set of routes that minimizes the total distance traveled by all vehicles combined.
2. **Complete Service:** Every customer must be visited exactly once by one vehicle.
3. **Depot Return:** Every vehicle route must start and end at the depot.

4. **Capacity Constraint:** The sum of demands of all customers on a single vehicle's route must not exceed the vehicle's capacity.
 5. **Time Window Constraint:** Each customer must be served within their specified time window (e.g., between 10:00 AM and 12:00 PM).
 6. **Service Time:** A fixed amount of time is spent at each customer location for unloading.
 7. **Vehicle Operating Hours:** Each vehicle has a maximum time it can be on the road for its route.
-

3. The Real-World Scenario: New Delhi Logistics

To make the problem realistic, we use actual locations in New Delhi, India.

- **Depot (1):** Connaught Place
- **Customers (10):** Various landmarks across the city.
- **Vehicles (3):** Each with a capacity of 15 units.

The specific data is as follows:

Index	Location Name	Latitude, Longitude	Demand (units)	Time Window (from 8:00 AM)
0	Depot: Connaught Place	(28.6315, 77.2167)	0	8:00 AM - 5:00 PM
1	India Gate	(28.6129, 77.2295)	1	9:00 AM - 10:00 AM
2	Red Fort	(28.6562, 77.2410)	1	11:00 AM - 12:00 PM

Index	Location Name	Latitude, Longitude	Demand (units)	Time Window (from 8:00 AM)
3	Qutub Minar	(28.5245, 77.1855)	2	1:00 PM - 2:00 PM
4	Lotus Temple	(28.5535, 77.2588)	4	1:00 PM - 2:00 PM
5	Humayun's Tomb	(28.5933, 77.2507)	2	10:00 AM - 11:00 AM
6	Chandni Chowk	(28.6507, 77.2319)	4	11:00 AM - 12:00 PM
7	Khan Market	(28.5940, 77.2225)	8	9:00 AM - 10:00 AM
8	Akshardham	(28.4744, 77.5026)	8	12:00 PM - 1:00 PM
9	Hauz Khas Village	(28.5522, 77.1950)	1	2:00 PM - 3:00 PM
10	Rohini	(28.7041, 77.1025)	2	8:00 AM - 10:00 AM

4. The Mathematical Foundation

The Haversine Formula

To calculate distances between real-world coordinates, we use the Haversine formula, which accounts for the Earth's curvature.

The distance d between two points with latitude ϕ and longitude λ is calculated as follows:

1. Calculate the intermediate value a :

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

2. Calculate the intermediate value c :

$$c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$$

3. Calculate the final distance d :

$$d = R \cdot c$$

Where:

- $\Delta\phi = \phi_2 - \phi_1$ (difference in latitude)
- $\Delta\lambda = \lambda_2 - \lambda_1$ (difference in longitude)
- R is the radius of the Earth (approx. 6,371 km).

5. Results and Conclusion

When the script is executed, it produces two outputs:

1. **A text summary in the console:** This details the optimized route for each vehicle, including the order of stops, the cumulative load, and the precise arrival time at each location, ensuring all time windows are respected.
2. **An interactive HTML map (delhi_routes_map_final.html):** This file, when opened in a browser, displays the routes visually on a map of New Delhi.

This classical solution is now complete. It is optimized, verifiable, and respects all the complex, real-world constraints we defined. It forms the perfect, high-quality benchmark for the next phase of the project: tackling the same problem with quantum computing algorithms.

